Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 1, Lab 1

Hardware: Program, Process, Thread, Core

1 Ideas

- Program — the file on disk (hello.c, a.out). It is not running yet.
- Process — one running copy of a program. Own memory. Two terminals running the same file = two processes.
- Thread — a worker *inside* a process. Threads of one process share that memory. • Core
— hardware that runs a thread. More cores means more workers at the same time.
- Socket / hyperthreading / cache — socket = one CPU chip (a laptop is usually 1). Hyper threading: one core can show as 2 logical processors in lscpu (Thread(s) per core). Cache: small fast memory on the chip (L1 closest, then L2, L3 often shared).

> A program becomes a process. A process can have many threads. Threads run on cores.

2 Commands

Terminal

```
lscpu
free -h
nproc
```

nproc is how many logical processors Linux will give you. Same idea as CPU(s) in lscpu.
Write these lines from *your* lscpu:

- CPU(s)
- Socket(s)
- Core(s) per socket
- Thread(s) per core
- L1d / L1i / L2 / L3

3 Reading lscpu

Example output

```
CPU(s): 8
Thread(s) per core: 2
Core(s) per socket: 4
Socket(s): 1
```

1 × 4 × 2 = 8 logical processors

Four real cores, hyperthreading on, one chip. Parallel code can use more than one of these at once. If nproc is 1, a parallel loop cannot show speedup on that machine.

4 A process from a program

C

```
# include < stdio .h >

int main ( void ) {
        printf ( " Hello from a process \ n " ) ;
        return 0;
}
```

Terminal

```
nano hello.c
gcc hello.c -o hello
 ./hello
```

nano: Ctrl+O, Enter (save), Ctrl+X (quit).
Forgot ./ → command not found.

This starts one process with one thread (the main thread).